graphDataProxy HTTP/TLS/JSON

Interface Design Description

Service ID: *"graphDataProxy"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `graphDataProxy` service of the GraphDBManager system.

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**
2.1 Interface registration . . . 3

3 Service Operations **4**
3.1 operation list-systems . . . 4
3.2 operation list-services . . . 4
3.3 operation get-data . . . 5
3.4 operation put-data . . . 5

4 Data Models **7**
4.1 struct ErrorMessage . . . 7
4.2 struct SenMLRecord . . . 7
4.3 struct ServiceListResponse . . . 7
4.4 struct SystemListResponse . . . 7
4.5 Primitives . . . 8

5 Go and Rust implementation variants **9**
5.1 Go implementation . . . 9
5.2 Rust implementation . . . 9

6 References **9**

7 Revision History **10**
7.1 Amendments . . . 10
7.2 Quality Assurance . . . 10

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the `graphDataProxy` service interface of the GraphDBManager system, which provides a cache of the latest SenML message per system and service, for consumers that cannot reach the producer directly.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `graphDataProxy` service interface

This document provides the Interface Design Description IDD to the *graphDataProxy – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `graphDataProxy` service interface in detail. Both in terms of its operations (Section 3) and its information model (Section 4).

2.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `graphdbmanager`, port 8467 and base path `/graphdbmanager/data-proxy`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`list-systems`	GET	`/systems`
`list-services`	GET	`/systems/{systemName}/services`
`get-data`	GET	`/systems/{systemName}/services/{serviceName}/data`
`put-data`	PUT	`/systems/{systemName}/services/{serviceName}/data`

3 Service Operations

The interfaces of the `graphDataProxy` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `graphDataProxy` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management, authorization or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

3.1 GET /graphdbmanager/data-proxy/systems

Operation: list-systems
Output: SystemListResponse

Returns the systems with cached data.

A successful call returns `200 OK` with the output in the body.

```
GET /graphdbmanager/data-proxy/systems HTTP/1.1
Host: graphdbmanager:8467
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 1: An list-systems invocation.

```
{
  "systems": [
    "string"
  ]
}
```

Listing 2: An list-systems response.

3.2 GET /graphdbmanager/data-proxy/systems/*{systemName}*/services

Operation: list-services
Input: String
Output: ServiceListResponse

Returns the services of a system with cached data.

Path parameter: *systemName*.

A successful call returns `200 OK` with the output in the body.

```
GET /graphdbmanager/data-proxy/systems/<systemName>/services HTTP/1.1
Host: graphdbmanager:8467
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An list-services invocation.

```
{
  "services": [
```

```
    "string"
  ]
}
```

Listing 4: An list-services response.

3.3 GET /graphdbmanager/data-proxy/systems/*{systemName}*/services/*{serviceName}*/data

Operation: get-data
Input: String
Output: Array<SenMLRecord>

Returns the cached SenML message of the system and service.
Path parameter: *systemName*, *serviceName*.
A successful call returns `200 OK` with the output in the body.

```
GET /graphdbmanager/data-proxy/systems/<systemName>/services/<serviceName>/data HTTP/1.1
Host: graphdbmanager:8467
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 5: An get-data invocation.

```
[
  {
    "bn": "TemperatureSensor/",
    "bt": 1792670400.0,
    "bu": "string",
    "n": "temperature",
    "u": "Cel",
    "v": 21.5,
    "vs": 21.5,
    "vb": true,
    "vd": "string",
    "t": 0.0
  }
]
```

Listing 6: An get-data response.

3.4 PUT /graphdbmanager/data-proxy/systems/*{systemName}*/services/*{serviceName}*/data

Operation: put-data
Input: Array<SenMLRecord>

Replaces the cached message of the system and service.
Path parameter: *systemName*, *serviceName*.
A successful call returns `204 No Content`.

```
PUT /graphdbmanager/data-proxy/systems/<systemName>/services/<serviceName>/data HTTP/1.1
Host: graphdbmanager:8467
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

[
  {
    "bn": "TemperatureSensor/",
    "bt": 1792670400.0,
```

```
    "bu": "string",
    "n": "temperature",
    "u": "Cel",
    "v": 21.5,
    "vs": 21.5,
    "vb": true,
    "vd": "string",
    "t": 0.0
  }
]
```

Listing 7: An put-data invocation.

4 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 4.5, which are used to represent things like hashes, identifiers and texts.

4.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	Error text, if the event is an error.
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

4.2 struct SenMLRecord

A SenML record (RFC 8428); a message is a JSON array of records.

Object Field	Value Type	Description
"bn"	String	SenML base name.
"bt"	Object	SenML base time (seconds).
"bu"	String	SenML base unit.
"bv"	Object	SenML base value.
"bs"	Object	SenML base sum.
"bver"	Object	SenML base version.
"n"	String	SenML name.
"u"	String	SenML unit.
"v"	Object	SenML numeric value.
"vs"	String	SenML string value.
"vb"	Boolean	SenML Boolean value.
"vd"	String	SenML data value (Base64).
"s"	Object	SenML sum.
"t"	Object	SenML time (seconds, relative to the base time).
"ut"	Object	SenML update time (seconds).

4.3 struct ServiceListResponse

Service names with data of a system.

Object Field	Value Type	Description
"services"	Array<String>	Service names with stored data.

4.4 struct SystemListResponse

List of systems.

Object Field	Value Type	Description
"systems"	Array<String>	System names with stored data.

4.5 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

5 Go and Rust implementation variants

5.1 Go implementation

No Go implementation folder for GraphDBManager is present in the designated primary Go tree. There is no port-specific executable, service binding or parity claim in this edition. A future port needs its own build, state model, security and interface qualification.

5.2 Rust implementation

No Rust implementation folder for GraphDBManager is present in the designated primary Rust tree. There is no port-specific executable, service binding or parity claim in this edition. A future port needs its own build, state model, security and interface qualification.

6 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “graphDataProxy – Service Description (GraphDBManager),” Eclipse Arrowhead, SD 5.3.0, 2026, graphDBManager-v5.3.0-graphDataProxy-SD.pdf.

7 Revision History

7.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-28	5.2.2	Document created from the corresponding DataManager document for the GraphDB-backed copy GraphDBManager.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged.	Claude Opus 5.5 (draft)
3	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

7.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	